Backbone block table 對照表

TWN_IOC_SET_TABLE 寫進 PL 的那 272 個字

依 tools/fuse_desctab.h（由 tools/gen_desctab.py 產生、不進版控，全文見第 7 節）與 rtl/fuse/twn_desctab.v 的欄位解碼和開機檢查整理

本表怎麼產生、怎麼驗證

所有數值都不是手抄的：腳本 gen_block_table.py 讀 tools/fuse_desctab.h 的 272 個字，欄位的位元位置直接從 rtl/fuse/twn_desctab.v 的 assign 敘述讀出，解碼後產生本文件的表格，並做下列核對，全部通過才輸出：

1. 把解出的欄位依同一套位元位置重新編碼，17 列 × 16 字全部和原字相同；這也證明所有「未用」位元在每一列都是 0。
2. 解出的通道數、尺寸、stride、residual、pack、stem、4 種參數的 offset / 字數 / 片上起點，和模型形狀檔 p1_manifest.json、權重索引 fuse_weights_index.json 逐列相同（17/17）。這兩個檔和 tools/fuse_desctab.h 一樣不進版控，由 docs/REBUILD.md 第 1 步產生。
3. 權重檔 offset 前後相接：每個區域的 offset 加上「字數 × 檔案中每字的位元組數」，補到 64 bytes 的倍數，等於下一個區域的 offset；每個區域的片上起點加字數不超過記憶體深度（17/17）。
4. 相鄰兩個 block（含 block 16 接回 block 0）在 4 個片上記憶體裡的區域都不重疊（17/17 對）。

1 Table 的形狀

項目	值
列數	17 列，一列一個 block（block 0 到 16），由 NBLK 暫存器決定跑幾列
每列	16 個 32-bit 字（64 bytes），稱為一個 descriptor
總大小	$17 \times 16 = 272$ 個字 = 1,088 bytes
PL 容量	20 列 × 16 字 = 320 個字（twn_desctab.v 的 NBLK_MAX = 20）
寫入位址	block b 的第 k 個字 → 暫存器位移 $0\text{x}1000 + 4\,(16b + k)$（相對於加速器暫存器基底；driver 的 R_DESC）
PL 內部索引	tab[{block, word}]，即 $16b + k$（9 位元）
來源	tools/gen_desctab.py 產生 tools/fuse_desctab.h（C 陣列，編進 app；不進版控，由 docs/REBUILD.md 第 1 步產生）
誰寫入	driver twn_set_table() 逐字 writel，再寫 NBLK、CHECK

	字 0	字 1	⋯	字 15	位址範圍
block 0	0x1000	0x1004	⋯	0x103C	0x1000–0x103F
block 1	0x1040	0x1044	⋯	0x107C	0x1040–0x107F
⋮					
block 16	0x1400	0x1404	⋯	0x143C	0x1400–0x143F

16 個字依用途分成三組：

- **形狀**（字 0、1、2、12）：通道數、尺寸、stride、打包方式、分段數，決定運算電路怎麼跑。
- **參數位置**（字 3–10）：4 種參數在 DDR 權重檔的哪裡、有多長、搬進片上記憶體的哪裡。
- **資料流向與 residual**（字 1 的 slot、字 13–15）：讀寫哪個 activation slot、residual 怎麼加。字 11（res_slot）有填值，但目前資料路徑不讀。

PL 用兩組暫存器讀 table：cur 存正在運算的 block 的全部 16 個字；curf 存正在預取權重的 block 的字 3–10（通常是下一個 block；每幀開頭是 block 0 本身）。開機檢查（CHECK）讀每列的字 0、1、7、8、9、10，以及下一列的字 1。

2　每個字的位元配置

左邊是 bit 31，右邊是 bit 0。單一位元的欄位用縮寫，縮寫列在該字下方。

字 0：通道數、stride、residual

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	S	c_out														c_mid												c_in			

R = `has_res` (bit 31)；S = `s_dw` (bit 30)

字 1：尺寸、pack、slot

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
out_slot				in_slot				pack_p			pack_e			w_in										h_in							

字 2：一列像素在 DDR 的位元組數

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
row_bytes_out																row_bytes_in															

字 3、4、5、6：4 種參數在權重檔的 byte offset (各佔整個字)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
字 3 `off_tern` ; 字 4 `off_dw` ; 字 5 `off_rqa` ; 字 6 `off_rqd`																															

字 7、8：4 種參數的字數

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
字 7 `n_dw` ; 字 8 `n_rqd`																字 7 `n_tern` ; 字 8 `n_rqa`													

字 9：三值權重、depthwise 權重的片上起點

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
未用，恆為 0								mb_dw																							

字 10：兩種 requant 的片上起點

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
未用，恆為 0								mb_rqd																							

字 11：residual 來源 slot (有填值，目前資料路徑不讀)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
未用，恆為 0																												res_slot			

字 12：分段數與打包旗標

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
未用，恆為 0																	P	E	pk0p		pk0e		T	nseg_p				nseg_e			

P = `msg_p` (bit 14)；E = `msg_e` (bit 13)；`pk0p` = `pack0_p` (bit 12:11)；`pk0e` = `pack0_e` (bit 10:9)；T = `stem` (bit 8)

字 13：residual 的 project 係數與右移量

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
未用，恆為 0								sh_res						mp_res (有號)																	

字 14：residual 的輸入係數

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
未用，恆為 0																mq_res (有號)															

字 15：residual 輸出夾限

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
hi_res (有號)																lo_res (有號)															

3　欄位對照總表

「誰讀」欄：**執行** = 正在運算的 block (`cur`)；**預取** = 權重搬移器讀正在預取的 block (`curf`)；**檢查** = 開機檢查；**無** = 資料路徑不讀。

字	位元	欄位	寬	編碼與單位	意義	誰讀
0	9:0	`c_in`	10	通道數	block 輸入的通道數	執行、檢查
0	19:10	`c_mid`	10	通道數	expand 1×1 之後的通道數 (stem 是 3×3 conv 的輸出)	執行、檢查
0	29:20	`c_out`	10	通道數	project 1×1 之後的通道數	執行、檢查
0	30	`s_dw`	1	0 = stride 1 · 1 = stride 2	3×3 depthwise 的 stride；2 時輸出高寬減半	執行

字	位元	欄位	寬	編碼與單位	意義	誰讀
0	31	`has_res`	1	0 / 1	是否把 block 輸入加回輸出 (residual)	執行
1	8:0	`h_in`	9	像素	輸入高度	執行、檢查
1	17:9	`w_in`	9	像素	輸入寬度	執行、檢查
1	20:18	`pack_e`	3	$\log_2$：0,1,2,3 → 1,2,4,8	expand 1×1 的打包方式	執行
1	23:21	`pack_p`	3	同上	project 1×1 的打包方式	執行
1	27:24	`in_slot`	4	slot 編號；`0xF` = 不讀 slot	從哪個 activation slot 讀；`0xF` 時從 `IN` 讀影像 (只有 block 0)	執行、檢查
1	31:28	`out_slot`	4	slot 編號	寫到哪個 activation slot	執行、檢查
2	15:0	`row_bytes_in`	16	bytes · 128 的倍數	輸入一列像素在 DDR 的跨距 = 通道 × 寬 · 補到 128 的倍數	執行
2	31:16	`row_bytes_out`	16	bytes · 128 的倍數	輸出一列的跨距	執行
3	31:0	`off_tern`	32	bytes · 64 的倍數	三值權重在權重檔裡的位置；PL 實際位址 = `W` 基底 + 此值	預取
4	31:0	`off_dw`	32	同上	depthwise 權重的位置	預取
5	31:0	`off_rqa`	32	同上	expand/project 的 requant 參數位置	預取
6	31:0	`off_rqd`	32	同上	depthwise 的 requant 參數位置	預取
7	15:0	`n_tern`	16	片上字數 (256 bit/字；檔案中 32 B/字)	三值權重的長度	預取、檢查
7	31:16	`n_dw`	16	片上字數 (576 bit/字；檔案中補到 80 B/字)	depthwise 權重的長度	預取、檢查
8	15:0	`n_rqa`	16	片上字數 (256 bit/字；檔案中 32 B/字)	expand/project requant 的長度	預取、檢查
8	31:16	`n_rqd`	16	片上字數 (448 bit/字；檔案中補到 64 B/字)	depthwise requant 的長度	預取、檢查
9	13:0	`mb_tern`	14	片上字位址 · 深度 8,160	三值權重放進片上記憶體的起點	執行、預取、檢查
9	23:14	`mb_dw`	10	片上字位址 · 深度 240	depthwise 權重的片上起點	執行、預取、檢查
10	12:0	`mb_rqa`	13	片上字位址 · 深度 600	expand/project requant 的片上起點	執行、預取、檢查
10	25:13	`mb_rqd`	13	片上字位址 · 深度 240	depthwise requant 的片上起點	執行、預取、檢查
11	3:0	`res_slot`	4	slot 編號	residual 來源 slot · 產生器一律填成 `in_slot` (與有無 residual 無關)。目前資料路徑不讀它：residual 的輸入列是 loader 讀 `in_slot` 時順便存下的；它只送到觀測輸出 `o_desc_rest` · 開機檢查也不看	無
12	3:0	`nseg_e`	4	從 1 起算	expand 權重分成幾段 (stem 是分段數 × kernel 列數)	執行
12	7:4	`nseg_p`	4	從 1 起算	project 權重分成幾段	執行
12	8	`stem`	1	0 / 1	是否為第一層 3×3 stride-2 conv	執行
12	10:9	`pack0_e`	2	$\log_2$；0 = 與 `pack_e` 相同	expand 第 0 段另用的 pack	執行
12	12:11	`pack0_p`	2	$\log_2$；0 = 與 `pack_p` 相同	project 第 0 段另用的 pack	執行
12	13	`msg_e`	1	0 / 1	expand 的混合分段旋轉	執行
12	14	`msg_p`	1	0 / 1	project 的混合分段旋轉	執行
13	17:0	`mp_res`	18	有號	residual：project 輸出的乘數	執行
13	23:18	`sh_res`	6	位元數	residual：相加後右移幾位	執行
14	17:0	`mq_res`	18	有號	residual：block 輸入的乘數	執行
15	15:0	`lo_res`	16	有號	residual 輸出的下限	執行
15	31:16	`hi_res`	16	有號	residual 輸出的上限	執行

residual 的算式 (`twn_res_add.v`)：p 是 project 的 int8 輸出，x 是 block 自己的輸入，

$$t = p \cdot \texttt{mp_res} + x \cdot \texttt{mq_res}, \qquad r = (t + 2^{\texttt{sh_res}-1}) \gg \texttt{sh_res}, \qquad \text{out} = \text{clamp}(r,\ \texttt{lo_res},\ \texttt{hi_res}).$$

$\gg$ 是算術右移；`sh_res` = 0 時不加捨入項也不移位，$r = t$。夾限後只保留低 8 位元。沒有 residual 的 block 也走同一個加法器，填單位值 `mp` = 1、`mq` = 0、`sh` = 0、夾限 $[-127, 127]$，輸出就等於 p。

位元組換算：片上記憶體每字的寬度不是 16 bytes 的倍數時，權重檔裡每字補到整數個 16-byte beat (depthwise 72 → 80 B，depthwise requant 56 → 64 B)，每個區域的起點對齊 64 bytes。

4　實例一：block 1 逐位元分解

MobileNetV2 的第二個 bottleneck：16 → 24 通道、expand 倍率 6、stride 2、沒有 residual。

16 個字的二進位

字	十六進位	二進位 (bit 31 → bit 0，每 4 位一組)
0	41818010	0100 0001 1000 0001 1000 0000 0001 0000
1	10090080	0001 0000 0000 1001 0000 0000 1000 0000
2	06000800	0000 0110 0000 0000 0000 1000 0000 0000
3	00000740	0000 0000 0000 0000 0000 0111 0100 0000
4	00000D40	0000 0000 0000 0000 0000 1101 0100 0000
5	00001100	0000 0000 0000 0000 0001 0001 0000 0000
6	000014C0	0000 0000 0000 0000 0001 0100 1100 0000
7	000C0030	0000 0000 0000 1100 0000 0000 0011 0000
8	000C001E	0000 0000 0000 1100 0000 0000 0001 1110
9	00391FB0	0000 0000 0011 1001 0001 1111 1011 0000
10	001C823A	0000 0000 0001 1100 1000 0010 0011 1010
11	00000000	0000 0000 0000 0000 0000 0000 0000 0000
12	00000011	0000 0000 0000 0000 0000 0000 0001 0001
13	00000001	0000 0000 0000 0000 0000 0000 0000 0001
14	00000000	0000 0000 0000 0000 0000 0000 0000 0000
15	007FFF81	0000 0000 0111 1111 1111 1111 1000 0001

逐欄位解碼

字	原始字	位元	欄位	欄位的位元	值	意義
0	41818010	9:0	c_in	0000010000	16	輸入 16 通道
		19:10	c_mid	0001100000	96	expand 後 96 通道 (倍率 6)
		29:20	c_out	0000011000	24	輸出 24 通道
		30	s_dw	1	1	stride 2，輸出 64×64
		31	has_res	0	0	無 residual
1	10090080	8:0	h_in	010000000	128	輸入高 128
		17:9	w_in	010000000	128	輸入寬 128
		20:18	pack_e	010	2	log2 編碼，pack = 4
		23:21	pack_p	000	0	log2 編碼，pack = 1
		27:24	in_slot	0000	0	讀 activation slot 0
		31:28	out_slot	0001	1	寫 activation slot 1
2	06000800	15:0	row_bytes_in	0000100000000000	2048	16 通道 × 128 像素 = 2048 B
		31:16	row_bytes_out	0000011000000000	1536	24 通道 × 64 像素 = 1536 B
3	00000740	31:0	off_tern	0x740	1856	三值權重在權重檔的位置 (byte)
4	00000D40	31:0	off_dw	0xD40	3392	depthwise 權重在權重檔的位置 (byte)
5	00001100	31:0	off_rqa	0x1100	4352	expand/project requant 在權重檔的位置 (byte)
6	000014C0	31:0	off_rqd	0x14C0	5312	depthwise requant 在權重檔的位置 (byte)
7	000C0030	15:0	n_tern	0000000000110000	48	48 字 (片上 256 bit/字)，檔案中佔 1536 B
		31:16	n_dw	0000000000001100	12	12 字 (片上 576 bit/字)，檔案中佔 960 B
8	000C001E	15:0	n_rqa	0000000000011110	30	30 字 (片上 256 bit/字)，檔案中佔 960 B
		31:16	n_rqd	0000000000001100	12	12 字 (片上 448 bit/字)，檔案中佔 768 B
9	00391FB0	13:0	mb_tern	01111110110000	8112	8112 + 48 = 8160，深度 8160
		23:14	mb_dw	0011100100	228	228 + 12 = 240，深度 240
10	001C823A	12:0	mb_rqa	0001000111010	570	570 + 30 = 600，深度 600
		25:13	mb_rqd	0000011100100	228	228 + 12 = 240，深度 240
11	00000000	3:0	res_slot	0000	0	等於本 block 的 in_slot；資料路徑不讀

字	原始字	位元	欄位	欄位的位元	值	意義
12	00000011	3:0	nseg_e	0001	1	分 1 段 (從 1 起算)
		7:4	nseg_p	0001	1	分 1 段 (從 1 起算)
		8	stem	0	0	不是 stem
		10:9	pack0_e	00	0	0 = 與 pack 相同
		12:11	pack0_p	00	0	0 = 與 pack 相同
		13	msg_e	0	0	混合分段旋轉：關
		14	msg_p	0	0	混合分段旋轉：關
13	00000001	17:0	mp_res	00000000000000001	1	單位值 1
		23:18	sh_res	000000	0	不移位
14	00000000	17:0	mq_res	00000000000000000	0	單位值 0
15	007FFF81	15:0	lo_res	1111111110000001	-127	輸出下限
		31:16	hi_res	0000000001111111	127	輸出上限

手算一個欄位：字 0 = `0x41818010`。

- `c_in` = bit 9:0 = `0x41818010 & 0x3FF` = `0x010` = 16。
- `c_mid` = bit 19:10 = `(0x41818010 >> 10) & 0x3FF` = `0x060` = 96。
- `c_out` = bit 29:20 = `(0x41818010 >> 20) & 0x3FF` = `0x018` = 24。
- bit 30 = 1 (最高的十六進位數字 4 = `0100`)，所以 stride 2；bit 31 = 0，所以沒有 residual。

這一列的兩個規律：

- 片上起點加字數剛好等於各記憶體深度 (8,112 + 48 = 8,160；228 + 12 = 240；570 + 30 = 600；228 + 12 = 240)，也就是放在記憶體的最上端；block 2 的字 9、10 全是 0，從最下端開始。從 block 1 起，奇數 block 靠上端、偶數 block 從 0 開始；block 0 的 `mb_tern` = 4,720，剛好接在 block 16 的三值權重 (從 0 起、4,720 字) 之後。規則是相鄰兩個 block 的片上區域不重疊：block k 運算時，block $k+1$ 的權重可以先預取進來，不會蓋到正在用的權重 (開頭的「相鄰 block 不重疊」核對)。16 → 0 也算相鄰是打包器 (`tools/pack_fuse_weights.py` 的 `_fits`) 的保守做法；目前的硬體在最後一個 block 不預取，block 0 的權重是每幀開頭、還沒有 block 在運算時才載入。
- 權重檔裡 4 個區域前後相接：`off_tern` 1,856 + 48 × 32 = 3,392 = `off_dw`；3,392 + 12 × 80 = 4,352 = `off_rqa`；4,352 + 30 × 32 = 5,312 = `off_rqd`；5,312 + 12 × 64 = 6,080 = block 2 的 `off_tern`。

5 實例二：block 2（有 residual）

16 → 24 之後的第一個 24 → 24 bottleneck：stride 1、有 residual，所以字 13、14 不再是單位值（字 11 照例等於 `in_slot` = 1）。

16 個字的二進位

字	十六進位	二進位（bit 31 → bit 0，每 4 位一組）
0	81824018	1000 0001 1000 0010 0100 0000 0001 1000
1	01088040	0000 0001 0000 1000 1000 0000 0100 0000
2	06000600	0000 0110 0000 0000 0000 0110 0000 0000
3	000017C0	0000 0000 0000 0000 0001 0111 1100 0000
4	00002240	0000 0000 0000 0000 0010 0010 0100 0000
5	00002800	0000 0000 0000 0000 0010 1000 0000 0000
6	00002D40	0000 0000 0000 0000 0010 1101 0100 0000
7	00120054	0000 0000 0001 0010 0000 0000 0101 0100
8	0012002A	0000 0000 0001 0010 0000 0000 0010 1010
9	00000000	0000 0000 0000 0000 0000 0000 0000 0000
10	00000000	0000 0000 0000 0000 0000 0000 0000 0000
11	00000001	0000 0000 0000 0000 0000 0000 0000 0001
12	00000021	0000 0000 0000 0000 0000 0000 0010 0001
13	0040B13E	0000 0000 0100 0000 1011 0001 0011 1110
14	0000B696	0000 0000 0000 0000 1011 0110 1001 0110
15	007FFF81	0000 0000 0111 1111 1111 1111 1000 0001

逐欄位解碼

字	原始字	位元	欄位	欄位的位元	值	意義
0	81824018	9:0	`c_in`	0000011000	24	輸入 24 通道
		19:10	`c_mid`	0010010000	144	expand 後 144 通道（倍率 6）
		29:20	`c_out`	0000011000	24	輸出 24 通道
		30	`s_dw`	0	0	stride 1，尺寸不變
		31	`has_res`	1	1	有 residual 加法
1	01088040	8:0	`h_in`	001000000	64	輸入高 64
		17:9	`w_in`	001000000	64	輸入寬 64
		20:18	`pack_e`	010	2	log2 編碼，pack = 4
		23:21	`pack_p`	000	0	log2 編碼，pack = 1
		27:24	`in_slot`	0001	1	讀 activation slot 1
		31:28	`out_slot`	0000	0	寫 activation slot 0
2	06000600	15:0	`row_bytes_in`	0000011000000000	1536	24 通道 × 64 像素 = 1536 B
		31:16	`row_bytes_out`	0000011000000000	1536	24 通道 × 64 像素 = 1536 B
3	000017C0	31:0	`off_tern`	0x17C0	6080	三值權重在權重檔的位置（byte）
4	00002240	31:0	`off_dw`	0x2240	8768	depthwise 權重在權重檔的位置（byte）
5	00002800	31:0	`off_rqa`	0x2800	10240	expand/project requant 在權重檔的位置（byte）
6	00002D40	31:0	`off_rqd`	0x2D40	11584	depthwise requant 在權重檔的位置（byte）
7	00120054	15:0	`n_tern`	0000000001010100	84	84 字（片上 256 bit/字），檔案中佔 2688 B
		31:16	`n_dw`	0000000000010010	18	18 字（片上 576 bit/字），檔案中佔 1440 B
8	0012002A	15:0	`n_rqa`	0000000000101010	42	42 字（片上 256 bit/字），檔案中佔 1344 B
		31:16	`n_rqd`	0000000000010010	18	18 字（片上 448 bit/字），檔案中佔 1152 B
9	00000000	13:0	`mb_tern`	00000000000000	0	0 + 84 = 84，深度 8160
		23:14	`mb_dw`	0000000000	0	0 + 18 = 18，深度 240
10	00000000	12:0	`mb_rqa`	0000000000000	0	0 + 42 = 42，深度 600
		25:13	`mb_rqd`	0000000000000	0	0 + 18 = 18，深度 240

字	原始字	位元	欄位	欄位的位元	值	意義
11	00000001	3:0	res_slot	0001	1	等於本 block 的 in_slot；資料路徑不讀
12	00000021	3:0	nseg_e	0001	1	分 1 段 (從 1 起算)
		7:4	nseg_p	0010	2	分 2 段 (從 1 起算)
		8	stem	0	0	不是 stem
		10:9	pack0_e	00	0	0 = 與 pack 相同
		12:11	pack0_p	00	0	0 = 與 pack 相同
		13	msg_e	0	0	混合分段旋轉：關
		14	msg_p	0	0	混合分段旋轉：關
13	0040B13E	17:0	mp_res	0010110001001111110	45374	$45374/2^{16} \approx 0.6924$
		23:18	sh_res	010000	16	右移 16 位
14	0000B696	17:0	mq_res	0010110110100101110	46742	$46742/2^{16} \approx 0.7132$
15	007FFF81	15:0	lo_res	1111111110000001	-127	輸出下限
		31:16	hi_res	0000000001111111	127	輸出上限

這一列的 residual：$\text{out} = \text{clamp}\left((45374\,p + 46742\,x + 2^{15}) \gg 16,\ -127,\ 127\right)$，即約 $0.6924\,p + 0.7132\,x$。

6　17 列總覽（解碼後）

pack 欄已從 $\log_2$ 換回實際值；dw stride 是 3×3 depthwise 的 stride（`s_dw`），block 0 的解析度減半由 stem 的 3×3 stride-2 conv 完成（256 → 128）；`in_slot` 的 F 代表 `0xF`（從 `IN` 讀影像）。block 16 寫 slot 0，BB2HD 開啟時 PL 改寫到 `OUT` 加 parity × 64 KB。

block	c_in	c_mid	c_out	輸入	dw stride	residual	pack_e	pack_p	讀 slot	寫 slot	mb_tern	stem
0	3	32	16	256×256	1	無	4	4	F	0	4720	是
1	16	96	24	128×128	2	無	4	1	0	1	8112	
2	24	144	24	64×64	1	有	4	1	1	0	0	
3	24	144	32	64×64	2	無	4	1	0	1	8060	
4	32	192	32	32×32	1	有	4	1	1	0	0	
5	32	192	32	32×32	1	有	4	1	0	1	8048	
6	32	192	64	32×32	2	無	4	1	1	0	0	
7	64	384	64	16×16	1	有	2	1	0	1	7776	
8	64	384	64	16×16	1	有	2	1	1	0	0	
9	64	384	64	16×16	1	有	2	1	0	1	7776	
10	64	384	96	16×16	1	無	2	1	1	0	0	
11	96	576	96	16×16	1	有	4	1	0	1	7248	
12	96	576	96	16×16	1	有	4	1	1	0	0	
13	96	576	160	16×16	2	無	4	1	0	1	6928	
14	160	960	160	8×8	1	有	1	1	1	0	0	
15	160	960	160	8×8	1	有	1	1	0	1	4720	
16	160	960	320	8×8	1	無	1	1	1	0	0	

7　原始 272 個字

block	字 0	字 1	字 2	字 3	字 4	字 5	字 6	字 7
0	01008003	0F4A0100	08000300	00000000	00000380	000004C0	00000640	0004001C
1	41818010	10090080	06000800	00000740	00000D40	00001100	000014C0	000C0030
2	81824018	01088040	06000600	000017C0	00002240	00002800	00002D40	00120054
3	42024018	10088040	04000600	000031C0	00003E40	00004400	00004980	00120064
4	82030020	01084020	04000400	00004E00	00005C00	00006380	00006A80	00180070
5	82030020	10084020	04000400	00007080	00007E80	00008600	00008D00	00180070
6	44030020	01084020	04000400	00009300	0000AD00	0000B480	0000BC80	001800D0
7	84060040	10042010	04000400	0000C280	0000F280	00010180	00010F80	00300180
8	84060040	01042010	04000400	00011B80	00014B80	00015A80	00016880	00300180
9	84060040	10042010	04000400	00017480	0001A480	0001B380	0001C180	00300180
10	06060040	01042010	06000400	0001CD80	00020980	00021880	00022780	003001E0
11	86090060	10082010	06000600	00023380	0002A580	0002BC00	0002D100	00480390
12	86090060	01082010	06000600	0002E300	00035500	00036B80	00038080	00480390
13	4A090060	10082010	05000600	00039280	00042C80	00044300	00045A00	004804D0
14	8A0F00A0	01001008	05000500	00046C00	00061A00	00063F80	00066280	00780D70
15	8A0F00A0	10001008	05000500	00068080	00082E80	00085400	00087700	00780D70
16	140F00A0	01001008	0A000500	00089500	000AE300	000B0880	000B3080	00781270

block	字 8	字 9	字 10	字 11	字 12	字 13	字 14	字 15
0	0004000C	001E1270	000F0140	0000000F	00000113	00000001	00000000	007FFF81
1	000C001E	00391FB0	001C823A	00000000	00000011	00000001	00000000	007FFF81
2	0012002A	00000000	00000000	00000001	00000021	0040B13E	0000B696	007FFF81
3	0012002C	00379F7C	001BC22C	00000000	00000021	00000001	00000000	007FFF81
4	00180038	00000000	00000000	00000001	00000021	00409D79	0000C697	007FFF81
5	00180038	00361F70	001B0220	00000000	00000021	00407269	0000BFF8	007FFF81
6	00180040	00000000	00000000	00000001	00000821	00000001	00000000	007FFF81
7	00300070	00301E60	001801E8	00000000	00000031	00407763	0000BD03	007FFF81
8	00300070	00000000	00000000	00000001	00000031	00404FD4	0000D31F	007FFF81
9	00300070	00301E60	001801E8	00000000	00000031	003C2FA1	000088AA	007FFF81
10	00300078	00000000	00000000	00000001	00000031	00000001	00000000	007FFF81
11	004800A8	002A1C50	001501B0	00000000	00002053	004095A7	0000BBCB	007FFF81
12	004800A8	00000000	00000000	00000001	00002053	00407B3E	00009E60	007FFF81
13	004800B8	002A1B10	001501A0	00000000	00002053	00000001	00000000	007FFF81
14	00780118	00000000	00000000	00000001	00000482	0040751E	0000AC97	007FFF81
15	00780118	001E1270	000F0140	00000000	00000482	0040B8C6	00003DFD	007FFF81
16	00780140	00000000	00000000	00000001	00000482	00000001	00000000	007FFF81

8 PL 的開機檢查（CHECK）

driver 寫 `CHECK` = 1 後，PL 從 block 0 走到 `NBLK` − 1，遇到第一個錯就停，把錯誤碼放在 `STATUS` bit 10:7、block 編號放在 bit 15:11。最後一列的「下一列」是 block 0。同一個 block 有多個條件成立時，只回報編號最小的那一個。

檢查沒過時，driver 的 `TWN_IOC_SET_TABLE` 回傳 `-EINVAL`，並在 dmesg 印出「descriptor table refused: error 碼 at block 號」（等不到檢查完成則回傳 timeout）；app 印出「SET_TABLE (refused: see dmesg)」後停止。在 PL 這一端，檢查還沒跑、正在跑、或沒通過時，backbone 的 START 一律被忽略，而且 START 這次寫入本身不會回報任何錯誤。

錯誤碼	名稱	條件	不擋下會怎樣
1	`E_SLOT_RANGE`	`in_slot` ≥ 2 且不是 `0xF`，或 `out_slot` ≥ 2	讀寫 activation 區以外的記憶體
2	`E_SLOT_SAME`	`in_slot` = `out_slot`	一邊讀一邊蓋掉自己的輸入
3	`E_SLOT_CHAIN`	本列 `out_slot` ≠ 下一列 `in_slot`（下一列是 `0xF` 時免檢）	下一個 block 讀到沒人寫的資料
4	`E_TERN_OVF`	`mb_tern` + `n_tern` > 8,160	三值權重超出片上記憶體
5	`E_DW_OVF`	`mb_dw` + `n_dw` > 240	depthwise 權重超出
6	`E_RQA_OVF`	`mb_rqa` + `n_rqa` > 600	requant 超出
7	`E_RQD_OVF`	`mb_rqd` + `n_rqd` > 240	depthwise requant 超出
8	`E_GEOM`	`c_in`、`c_mid`、`c_out`、`h_in`、`w_in` 任一為 0	運算長度為 0

檢查**看不到**的：

- 字 3–6 的 offset 是否指向權重檔裡對的位置，字 7–8 的字數是否等於權重檔裡實際的長度；
- 相鄰 block 的片上區域是否重疊（它只比較起點加長度和記憶體深度）；
- 字 0 的 `s_dw`、`has_res`，字 1 的 `pack_e`、`pack_p`，字 2 的跨距是否和通道數相符，字 11–15 的內容；
- `NBLK` 是否 ≤ 20：`NBLK` 是 5 位元、PL 不和 table 容量比較，driver 接受到 31；`NBLK` = 0 時檢查走不到結束條件。

offset、字數與片上位置由 `tools/pack_fuse_weights.py` 決定（相鄰不重疊是它的 `_fits` 規則），`tools/gen_desctab.py` 照它寫出的權重索引抄進 table，並用 `assert` 檢查 slot、容量與欄位寬度。所以 table 和權重檔必須出自同一次打包。